

EVENTFORCE MANAGEMENT SYSTEM

PHASE 2

Backend Development & Configurations

Objects, relationships, data quality, automation, approvals and Apex business logic

Elaborated Salesforce Implementation Document

Prepared as a detailed, project-specific submission document

1. Phase Overview

Build the functional engine of EventForce so event operations can be stored, validated, automated and processed consistently.

2. Backend Development Strategy

The backend phase converts the Phase 1 requirements into Salesforce configuration. The implementation combines declarative features—objects, fields, relationships, validation rules, formula fields, Flows and Approval Processes—with Apex for cross-record business rules and scheduled processing.

3. Salesforce Environment Setup

- Development is carried out in a Salesforce Developer Edition environment.
- The Salesforce Setup and Object Manager are used for the configuration work.
- Developer Console is used for Apex classes and triggers.
- Custom objects are deployed and exposed for use in the EventForce application.

4. Custom Object Model

Object	Important fields / role
Event__c	Event Name, Event Date, Event Type, Event Status, Event Budget; central transaction object.
Client__c	Client Name, Email, Phone, Address, Country, City.
Vendor__c	Vendor Name, Email, Phone, Service Type, Status.
Venue__c	Venue Name, Address, Location (URL), Capacity, Availability Status.
Feedback__c	Feedback ID / Auto Number, Rating, Comments; related to Event and Client.
EventVendor__c	Junction object connecting Event and Vendor through two master-detail relationships.

5. Relationship Design

Parent / Child relationship	Type	Purpose
Event → Client	Lookup	Associates an event with the responsible client.
Event → Venue	Lookup	Associates an event with its venue.
Feedback → Event	Lookup	Connects feedback to the relevant event.
Feedback → Client	Lookup	Connects feedback to the relevant client.
EventVendor → Event	Master-Detail	Makes Event the master side of the junction.
EventVendor → Vendor	Master-Detail	Connects each junction record to a vendor.

6. Data Quality Configuration

- Validation Rules are used to stop invalid data from being accepted when business conditions are not met.
- Event date and required relationship checks support consistent event records.

- Email-format validation supports cleaner client/vendor contact data.
- Formula Fields can derive values without requiring users to calculate them manually.
- The documented Event Budget formula maps Event Type values to standard budget amounts.

Documented budget formula logic: CASE(TEXT(Event_Type__c), "Wedding", 50000, "Corporate", 30000, "Birthday", 10000, "Anniversary", 20000, "Festival", 60000, "Concert", 40000, "Other", 15000, 0).

7. Client Reminder Flow

- The documented automation is named Client Reminder – 3 Days Before.
- It is implemented as a record-triggered Flow with a scheduled path.
- The Flow is intended to act on appropriate event records and support timely client communication.
- The scheduled timing separates the event-record change from the future reminder action.
- Testing should confirm that qualifying events receive the intended reminder and non-qualifying records do not.

8. Event Cancellation Approval Process

- Approval Process name: Event_Cancellation_Process.
- Entry condition: Event Status equals Pending Cancellation.
- Initial submitter: Event Owner.
- Approval can be assigned to a manager/specific approver according to the configured process.
- Initial submission action sends the cancellation request notification.
- Final approval updates Event Status to Canceled and sends an approval notification.
- Final rejection sends a rejection notification and updates the status according to the configured rejection action.

Workflow stage	Configured behavior
Submission	Event owner submits a Pending Cancellation record.
Review	Assigned approver reviews event details.
Approval	Status is updated to Canceled and notification is sent.
Rejection	Rejection notification is sent and status is updated to the configured value.

9. Apex Business Logic

Component	Purpose
VenueStatusHelper	Supports venue availability/status handling.
EventTrigger13	After-insert/update event processing.
PreventDoubleBooking	Before insert/update check for venue/date conflicts.
BatchCompleteEvents	Batch processing to mark past events as Completed.
ScheduleCompleteEvents	Schedulable wrapper that executes the batch process.

- PreventDoubleBooking collects venue IDs from Trigger.new, queries relevant existing events, groups them by venue and checks for the same event date.
- If an existing event already uses the same venue on the same date, the trigger adds an error and prevents the conflicting save.
- BatchCompleteEvents queries past events whose status is not Completed and updates them to Completed.
- ScheduleCompleteEvents invokes the batch using Database.executeBatch.

10. Scheduled Event Completion

- The documented job is named Daily Event Completion.
- The schedulable class is ScheduleCompleteEvents.
- The batch class is BatchCompleteEvents.
- The source configuration schedules the job for a recurring evening run so past events can be marked Completed.
- Scheduling is configured through Setup → Apex Classes → Schedule Apex.

11. Backend Verification Plan

Test	Expected behavior
Valid Event	Record saves successfully when required data is correct.
Invalid event data	Validation rule blocks the save.
Reminder scenario	Qualifying event produces the intended reminder action.
Cancellation	Pending Cancellation enters the approval workflow.
Double booking	Conflicting venue/date save is rejected.
Past event	Scheduled/batch processing changes eligible status to Completed.

12. Phase Outcome

Phase 2 transforms the EventForce design into a functional backend. Declarative automation handles standard workflow needs, while Apex addresses cross-record and scheduled processing requirements. The resulting backend becomes the foundation for the Lightning UI, reporting, migration and testing phases.

Implementation Evidence

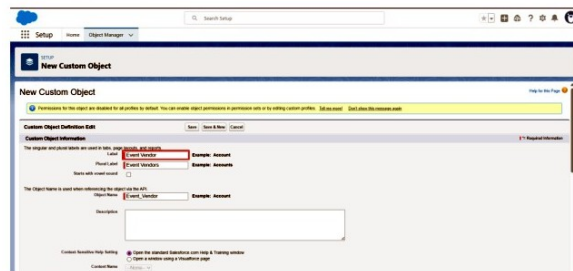
The following selected visuals are included as supporting implementation evidence. The document layout and narrative are reorganized for a distinct phase-wise submission.

Create Many-to-Many Relationship (Event ↔ Vendor)

Salesforce doesn't allow direct many-to-many, so we create a **junction object** (EventVendor__c) :

Go to Object Manager → Create New Custom Object

- Label: Event Vendor
- Object Name: EventVendor__c

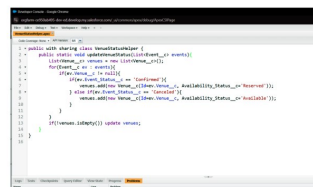


From Setup, go to Object Manager, Select Event Vendor Object

Backend / automation configuration evidence.

```
public with sharing class VenueStatusHelper {  
    public static void updateVenueStatus(List<Event__c> events){  
        List<Venue__c> venues = new List<Venue__c>();  
        for(Event__c ev : events){  
            if(ev.Venue__c != null){  
                if(ev.Event_Status__c == 'Confirmed'){  
                    venues.add(new Venue__c(id=ev.Venue__c, Availability_Status__c='Reserved');  
                } else if(ev.Event_Status__c == 'Canceled'){  
                    venues.add(new Venue__c(id=ev.Venue__c, Availability_Status__c='Available');  
                }  
            }  
        }  
        if(venues.isEmpty()) update venues;  
    }  
}
```

5. Save the class using CTRL+S



New Create Trigger For this Class :

Apex implementation evidence.

Appendix — Phase Checklist

- All six custom objects configured.
- Lookup and master-detail relationships configured.
- Validation and formula logic added.
- Reminder Flow configured.
- Cancellation Approval Process configured.
- Apex trigger/classes and scheduled processing configured.
- Backend test scenarios documented.

EventForce Management System | Salesforce Implementation